

CASUALTY & DISASTER INSURANCE MANAGEMENT SYSTEM

1. Project Overview

A full-stack enterprise insurance platform that automates the complete lifecycle of casualty and disaster insurance — from policy creation and policy sales, through claim submission, geographic disaster verification, fraud detection, investigation, and payment. Built on .NET 8 Clean Architecture, SQL Server, and Angular with Tailwind CSS.

What Makes This System Different from Typical Projects

✓ Geographic disaster verification — incident location & date cross-checked against real disaster event data

✓ Fraud detection engine — fraud scoring engine that auto-flags suspicious claims

✓ 9-stage claim state tracking — not just pending/approved/rejected

✓ 5 specialized roles — each with distinct workflow responsibilities and data access scope

✓ Auto-approval logic — low-value, low-risk claims bypass the investigation stage automatically

✓ Clean Architecture — Domain, Application, Infrastructure, API separation

Tech Stack

Layer	Technology	Purpose
Backend API	.NET 8 / ASP.NET Core Web API	RESTful endpoints, business logic, authentication
ORM	Entity Framework Core 8	DB access, Code-First migrations
Database	Microsoft SQL Server	Primary relational data store
Authentication	ASP.NET Core Identity + JWT	Secure role-based authentication
Frontend	Angular (Standalone)	SPA, reactive forms, route guards
Styling	Tailwind CSS	Utility-first component design
Architecture	Clean Architecture	Domain / Application / Infrastructure / API

2. System Roles

All five roles share a single Users table with a Role enumerator. Extended by a Customers table for customer-specific profile fields. Role-based access enforced via JWT claims on the backend and canActivate route guards on the frontend.

Role	Core Responsibilities	Claim Authority
Admin	Create policy products, manage all users, system analytics	None
Sales Agent	Sell policies to customers, track commissions, assist with claims, checks whether customer is paying premium or not — linked to policy until expiry	None
Claims Manager	Review new claims, assign agents, review investigation reports, make final decision, trigger payment	Claim status and amount credit to customers
Claims Agent	Investigate assigned claims, review documents + fraud + geo reports, upload investigation report	Report only — no approval
Customer	Buy policies, submit claims, upload documents, track claim status in portal	Submission only

Role Separation Rules

Sales Agent is tied to the POLICY (permanent until expiry) — not to the claim. Claims Agent is tied to the CLAIM (temporary, per-claim lifecycle). Admin has zero operational involvement in claims workflow.

3. Data Models

3.1 Users + Customers

```
Users: Id | FirstName | LastName | Email (unique) | PasswordHash
      Role (0=Admin,1=SalesAgent,2=ClaimManager,3=ClaimAgent,4=Customer)
      IsActive | CreatedAt | UpdatedAt

Customers (extends Users 1:1): UserId(PK,FK) | SalesAgentId(FK→Users)
      Address | City | State | ZipCode | DateOfBirth
```

3.2 PolicyProducts (Admin creates — insurance plan templates)

```
PolicyProducts: Id | Name | PolicyType(Flood,Earthquake,Fire,Hurricane,Wildfire...)
      BasePremium | MaxCoverageAmount | DeductibleAmount | WaitingPeriodDays
      CreatedByAdminId(FK) | CreatedAt | IsActive
```

3.3 Policies (Sales Agent sells to Customer)

```
Policies: Id | PolicyNumber(unique) | PolicyProductId(FK) | CustomerId(FK)
      SalesAgentId(FK) | CoverageAmount | PremiumAmount
      StartDate | EndDate | Status(Active,Expired,Cancelled) | CreatedAt
```

3.4 Claims (Core operational table)

```
Claims: Id | ClaimNumber(unique) | PolicyId(FK) | CustomerId(FK)
      ClaimManagerId(FK) | ClaimAgentId(nullable FK) ← temporary assignment
      ClaimAmount | ApprovedAmount(null) | IncidentDate | Description
      IncidentLatitude | IncidentLongitude ← geo verification
      Status(enum 0-9) | FraudScore | IsFraudFlagged | FraudReason
      GeoVerificationStatus(Pending,Verified,Unverified,Inconclusive)
      GeoVerificationNote | SubmittedAt | AssignedAt
      InvestigationCompletedAt | ClosedAt
```

3.5 Supporting Tables

Table	Key Columns	Relationship
ClaimDocuments	ClaimId(FK), FileName, FileUrl, DocumentType, UploadedByUserId, UploadedAt	Claim 1→Many
FraudAssessments	ClaimId(FK), Score, RiskLevel(0-3), RulesTriggered(JSON), CreatedAt	Claim 1→Many
Payments	ClaimId(FK UNIQUE), Amount, PaymentStatus, TransactionReference, ProcessedAt	Claim 1→0..1
SalesCommissions	PolicyId(FK), SalesAgentId(FK), CommissionAmount, PaidStatus, CreatedAt	Policy 1→1
AuditLogs	UserId(FK), Action, EntityType, EntityId, OldValue(JSON), NewValue(JSON), CreatedAt	All entities

4. Entity Relationships & Cardinality

Entity A	Entity B	Cardinality	FK Location
Admin	PolicyProduct	1 : Many	CreatedByAdminId on PolicyProducts
SalesAgent	Customer	1 : Many	SalesAgentId on Customers
SalesAgent	Policy	1 : Many	SalesAgentId on Policies
Customer	Policy	1 : Many	CustomerId on Policies
PolicyProduct	Policy	1 : Many	PolicyProductId on Policies
Customer	Claim	1 : Many	CustomerId on Claims
Policy	Claim	1 : Many	PolicyId on Claims
ClaimManager	Claim	1 : Many	ClaimManagerId on Claims (mandatory)
ClaimAgent	Claim	1 : Many (temp)	ClaimAgentId on Claims (nullable)
Claim	ClaimDocument	1 : Many	ClaimId on ClaimDocuments
Claim	FraudAssessment	1 : Many	ClaimId on FraudAssessments
Claim	Payment	1 : 0..1	ClaimId UNIQUE on Payments

5. Cardinality

Claims move through 9 enforced states. The backend rejects any transition not in the allowed set — this is not a CRUD flag, it is a real workflow engine.

#	Status	Triggered By	Notes
0	Submitted	Customer	Policy validated at submission time
1	UnderManagerReview	System	Auto-set after submission, notifies Manager
2	AssignedToAgent	Claims Manager	Sets AssignedAt timestamp; ClaimAgentId filled
3	InvestigationCompleted	Claims Agent	Report uploaded, sets InvestigationCompletedAt
4	Approved	Claims Manager	Full ClaimAmount approved
5	PartiallyApproved	Claims Manager	ApprovedAmount < ClaimAmount

#	Status	Triggered By	Notes
6	Rejected	Claims Manager	Reason stored, no payment triggered
7	Paid	Payment Service	Triggered on Approved/PartiallyApproved
8	Closed	System	Final state — Paid or Rejected path complete
9	FlaggedForFraud	Fraud Engine / Manager	Blocks normal progression pending review

Auto-Approval Rule (skips AssignedToAgent stage)

Condition: ClaimAmount < 5% of CoverageAmount AND FraudScore < 20 AND GeoStatus = Verified AND ≥1 document uploaded

Result: Status jumps from UnderManagerReview → Approved directly. Reduces agent workload on routine small claims.

6. System Workflow

6.1 Policy Sales Flow

Step	Actor	Action
1	Admin	Creates a PolicyProduct template (e.g., Flood Insurance Plan A) — sets type, premium, max coverage, deductible, waiting period
2	Sales Agent	Selects an active PolicyProduct, assigns it to a Customer → Policy record created with PolicyNumber, StartDate, EndDate, SalesAgentId
3	System	Calculates premium, creates SalesCommission record for the agent, Policy.Status = Active after WaitingPeriodDays

6.2 Claim Submission Flow

Step	Actor	Action
1	Customer	Submits claim: selects policy, enters IncidentDate, location (lat/lng), ClaimAmount, Description — uploads documents
2	System	Validates: Policy Active? IncidentDate within policy period? Waiting period elapsed? Coverage type matches?
3	System	Runs Geographic Disaster Verification — queries disaster events DB for matching type/location/date. Sets GeoVerificationStatus.
4	System	Runs Fraud Detection Engine — scores claim 0–100, sets FraudScore, IsFraudFlagged. Checks Auto-Approval Rule.
5	System	Status = Submitted → UnderManagerReview. Manager dashboard notified. If auto-approval triggers → Approved directly.

6.3 Investigation & Decision Flow

Step	Actor	Action
1	Claims Manager	Reviews claim, fraud score, geo-verification result, uploaded documents
2	Claims Manager	Assigns a Claims Agent → Status = AssignedToAgent, AssignedAt timestamp set
3	Claims Agent	Investigates — reviews all documents, fraud report, geo result — uploads formal investigation report
4	Claims Agent	Marks complete → Status = InvestigationCompleted, InvestigationCompletedAt set
5	Claims Manager	Reviews agent report → Decision: Approve / Partially Approve / Reject (with reason + amount)
6	System	If approved → Payment record created, PaymentStatus = Pending → Completed → Status = Paid → Closed

7. Fraud detection

Runs automatically on every claim submission. Each rule carries a fixed maximum score contribution. All 10 rules together sum to exactly 100 — the score represents the percentage of maximum fraud signals triggered. Multiple assessments are stored in the FraudAssessments table as the claim progresses.

Rule	Trigger Condition	Max Score	Severity
IncidentDateBeforePolicyStart	IncidentDate < Policy.StartDate	20	Critical
ClaimAmountExceedsCoverage	ClaimAmount > CoverageAmount	18	Critical
WaitingPeriodViolation	IncidentDate within WaitingPeriodDays of StartDate	15	Critical
MultipleClaimsOnSamePolicy	2+ existing claims on same Policy	13	High
PreviousFraudHistory	Customer has prior FlaggedForFraud claim	12	High
HighValueClaim	ClaimAmount > 80% of CoverageAmount	10	High
GeoVerificationFailed	GeoVerificationStatus = Unverified	8	Medium
RecentPolicyHolder	Policy age < 90 days at claim time	7	Medium
NoDocumentsUploaded	Zero documents at submission	4	Low
RoundNumberClaim	ClaimAmount is a suspiciously round figure	3	Low
TOTAL	All rules triggered simultaneously	100	—

Score	Risk Level	System Action
0 – 19	Low	Normal flow. Eligible for auto-approval if other conditions met.
20 – 49	Medium	Flagged in UI for Manager attention. Proceeds normally.
50 – 74	High	IsFraudFlagged = true. Manager must review before assigning agent.
75 – 100	Critical	Status → FlaggedForFraud immediately. Blocked until reviewed.

8. Geographic Disaster Verification

When a customer submits a claim, the system cross-references the reported incident location and date against a disaster events database. A flood claim in a location with no recorded flood event is a major fraud signal — this is standard practice in real insurance.

Step	Description
1. Capture	Customer provides IncidentLatitude, IncidentLongitude, IncidentDate, and PolicyType (e.g., Flood)
2. Query	System queries disaster events source — matching disaster type, date range (± 7 days), geographic radius (~50km)
3. Verified	Match found → GeoVerificationStatus = Verified. Supports claim legitimacy.
4. Unverified	No event found in area/timeframe → GeoVerificationStatus = Unverified. FraudScore +20. Manager alerted.
5. Inconclusive	Insufficient data to confirm → GeoVerificationStatus = Inconclusive. Flagged for manual review.
6. Stored	GeoVerificationStatus + GeoVerificationNote saved on Claim. Visible to Agent and Manager.

9. Backend Architecture & Frontend

9.1 Clean Architecture Layers

Layer	Project	Contains
Domain	InsuranceSystem.Domain	Entities, enums, value objects — zero external dependencies
Application	InsuranceSystem.Application	Interfaces, DTOs, CQRS commands/queries, validators, service contracts
Infrastructure	InsuranceSystem.Infrastructure	EF Core DbContext, repositories, fraud engine, geo service, identity, email
API	InsuranceSystem.API	Controllers, JWT middleware, Swagger, request/response mapping

9.2 Key API Endpoints

Method	Endpoint	Role	Description
POST	/api/auth/login	Public	Returns JWT + role
POST	/api/policy-products	Admin	Create product template
POST	/api/policies	SalesAgent	Sell policy to customer
POST	/api/claims	Customer	Submit claim + trigger fraud + geo check
PUT	/api/claims/{id}/assign-agent	Manager	Assign Claims Agent
PUT	/api/claims/{id}/investigation-complete	ClaimAgent	Upload report, complete investigation
PUT	/api/claims/{id}/decision	Manager	Approve / Partial / Reject
GET	/api/claims/{id}/fraud-report	Agent/Manager	Full fraud assessment details
GET	/api/audit-logs	Admin	System-wide audit trail

9.3 Angular Frontend

Module	Route	Accessible By	Key Features
Auth	/auth	Public	Login, customer registration
Admin	/admin	Admin	User mgmt, products, audit logs, dashboard
Sales	/sales	Sales Agent	Customer list, sell policy, commissions
Claims	/claims	Manager + Agent	Claims queue, assign, report, decision
Customer Portal	/portal	Customer	My policies, submit claim, doc upload, status

JWT Interceptor attaches Bearer token to every request. Role-based canActivate guards on all protected routes. Reactive Forms with validators on all inputs. Real-time claim status polling on customer portal.

10. How This Project Differs from Others

Feature	Typical Project	This System
Claim workflow	Pending → Approved / Rejected	9-stage enforced state machine with invalid transition protection
Fraud detection	Not present	10-rule weighted engine, risk levels, auto-flagging, stored assessments
Geographic verification	Not present	Incident location + date cross-referenced against disaster event database
Role architecture	Admin + User (2 roles)	5 specialized roles, each scoped to specific workflow stages
Claims Agent model	Single approver	Temporary adjuster — assigned per claim, no approval authority
Auto-approval	Not present	Intelligent rule skips investigation for low-value, low-risk, geo-verified claims
Audit trail	Not present	Every action logged with actor, timestamp, old/new values in JSON
Data model	Simple FK joins	Fully normalized, cardinality-precise, Clean Architecture ready
Commission tracking	Not present	SalesCommissions table tracks per-policy agent earnings
Architecture	MVC or flat layering	Clean Architecture: Domain / Application / Infrastructure / API

Summary

This is not a CRUD demo. It models how a real mid-sized insurance company operates end to end. The combination of geographic disaster verification + weighted fraud scoring + enforced state machine + role-separated workflow puts this system well beyond what is typically built in academic or portfolio insurance projects. Built on production-grade .NET 8 Clean Architecture + SQL Server + Angular 17 — fully scalable.